

PROJECT MANAGEMENT

[CS-604]

UNIT-2

Syllabus -

Software management Process Framework:
life cycle phases - inception, elaboration,
construction & training phase.

Artifacts of the process - the artifact
sets, management artifacts, engineering
artifacts, pragmatics artifacts.

model based software architectures.

Workflows of the process. Checkpoints
of the process.

SOFTWARE MANAGEMENT PROCESS

The software management process is the set
of activities used to plan, organize, monitor
& control software development so that the
project is completed on time, within
budget & with required quality.

Objective of Software Management

- Deliver software on time
- Stay within budget
- Maintain high quality
- Efficient use of resources
- Handle risks effectively.

SOFTWARE PROCESS FRAMEWORK

A framework defines the structure of software development.

Elements:

1. Phases
2. Artifacts
3. Workflows
4. Checkpoints

LIFE CYCLE PHASES

[May, 2022] [May, 2023]
[June, 2024]

Life cycle Phases are the stages of a project from beginning to end.

They describes "how a project starts, runs & finishes.

Software development is divided into 4 main phases:

- 1) Inception phase
- 2) Elaboration phase
- 3) Construction phase
- 4) Transition phase

1) INCEPTION PHASE -

The Inception Phase is the first & most important phase in the software development life cycle.

It focuses on understanding what to build & whether it is worth building.

Objective of Inception Phase-

1) Define Project Scope-

- The objective is to clearly define what the project will include & what it will not.
- This helps avoid confusion & prevents unnecessary expansion of the project later.

2) Identify Key Requirements

- It focuses on understanding the main needs of users & stakeholders at a high level.
- This ensures the system is built according to user expectations.

3) Analyze Feasibility

- The goal is to check whether the project is technically possible, economically ~~viable~~ viable & practical to implement.
- It helps in deciding whether to continue the project.

4) Estimate Cost & Time

This objective provides a rough estimate of the budget, time & resources required.

5) Identify Risk

- It involves identifying possible risks or problems that may occur during the project.
- Early identification helps in reducing future issues.

Deliverables (Outputs)

At the end of the inception Phase, these documents are prepared:

- Project Proposal
- Business Case Document
- Initial Risk List
- High-level Requirement Document
- Rough Cost & Schedule Estimate

Importance of Inception Phase

- Provides clear direction for the project.
- Prevents waste of time & resources.
- Helps in decision - making (go / no-go).

- Reduces chances of project failure.

4) ELABORATION PHASE -

It is the most critical phase among the four phases.

The Elaboration Phase is the stage where the project requirements are analyzed in detail, system architecture is designed & major risks are addressed before actual development begins.

Objectives of Elaboration Phase

1) Refine Requirements

- Convert high-level requirements into detailed & clear requirements.
- Remove ambiguity & confusion.

2) Design System Architecture

Create a blueprint of the system.

Decide structure, components & interactions.

3) Risk Reduction

- Identify & resolve critical risks ~~only~~ early.
- focus on technical challenges & uncertainties

4) Establish Project Plan

Prepare a detailed plan for:

- Time
- Cost
- Resources

5) Validate Project Feasibility

Ensure that the project is still practical & achievable after deeper analysis.

Deliverables (Output)

At the end of the Elaboration Phase, the following are ready:

- Software Architecture Document (SAD)
- Detailed Requirement Specification (SRS)
- Updated Risk list
- Project Plan
- Prototype (if required)

Importance of Elaboration Phase -

- Provides a strong foundation for development.
- Reduces chances of failure due to unclear requirements.
- Ensures technical feasibility.
- Helps avoid costly changes later.

3) CONSTRUCTION PHASE

The Construction Phase is the stage where the software is developed, coded, integrated & tested based on the design and requirements prepared in earlier phases.

Objective of Construction Phase

1. Develop the system

- Convert design into working software through coding.
- Implement all features & functionalities.

2. Complete Remaining Requirements

Ensures all functional & non-functional requirements are implemented.

3. System Integration

Combine different modules into a complete system.

4. Testing & Quality Assurance

Perform:

- Unit testing → Each module is tested individually
- Integration testing → Combine module step-by-step
- System testing → Test the complete system.

5. Prepare for Deployment

Make the system ready for delivery to users.

Deliverables (Output)

At the end of the Construction Phase:

- Working software Product.
- Test Cases & Test Reports
- Technical Documentation.

- User manuals (basic)

Importance of Construction Phase-

- Converts plans into reality.
- Ensures software quality through testing.
- Produces a fully functional system.
- Prepares the product for final delivery.

4) TRANSITION PHASE

The transition Phase is the last phase.

The transition Phase is the stage where the developed software is deployed, tested in the real environment & handed over to users, along with necessary support & training.

Objectives of Transition Phase

1. Deploy the System

- Install the software in the actual user environment.
- make it available for real users.

d. User Acceptance

- Ensure the system meets user requirements & expectations.
- Conduct acceptance testing.

3. Fix Remaining Issues

- Identify & correct:
 - Bugs
 - Errors
 - Performance Issues.

4. User Training

- Train users on how to use the system effectively.

5. Ensure Smooth Operation

- Make sure the system runs properly without major problems.

Deliverables (Outputs)

At the end of the Transition Phase:

- Final Software Product (Delivered System)

- User manuals & Documentation
- Training materials
- Maintenance Plan.

Importance of Transition Phase -

- Ensures successful delivery of the project.
- Confirms customer satisfaction.
- Helps users adapt to the system easily.
- Reduces post-deployment issues.

ARTIFACTS OF THE PROCESS [May, 2023] [May, 2024]

Artifacts are the documents, models & outputs created during software development.

They show what work is done in each step of the process

In Conventional (Traditional) Process -

Software is developed in a step-by-step (sequential) way & at each step, a specific artifact is created:

1. Build the Requirements-

- First, we understand what the user wants.
- we collect all needs & features.

2. Create Design Model-

- After requirements, we plan how the system will work.
- Design should match the requirements.

3. Compile and Test-

- Then developers write code (implementation)
- After coding, software is tested to remove errors.

4. Works for Small Projects-

- This method is okay for:
 - Small Projects
 - Simple Systems
- Because everything is closely connected & easy to manage.

5. Not Suitable for Modern Systems -

• Today's software is :

- Very Complex
- Made of many parts (components)
 - Some are customer - built
 - Some are reused
 - Some are ready - made

So this simple step-by-step method doesn't work well now.

THE ARTIFACTS SET [May, 2023] [May, 2024]

An Artifact Set is a group of related artifacts (documents, models, code, etc.) that represent a particular aspect of the software system.

Main Types of Artifact sets -

There are 4 major artifact sets in software project management:

1. Management Artifact Set

These artifacts are used to plan, monitor & control the project.

It Includes:

- Project Plan
- Schedule
- Cost estimation
- Risk list
- Progress reports.

It Ensure project is completed on time and within budget.

2. Engineering Artifact Set

These artifacts describe the technical development of the system.

It Includes:

- Requirements documents
- Design models (UML diagrams)
- Source code
- Test cases.

The Purpose is to show how the system is built & works.

3. Deployment Artifact set

These artifact are related to delivery and installation of the system.

It includes:

- Installation files
- user manuals
- Release notes
- Training materials.

The purpose is to help users use and install the software.

4. Environment Artifact set

These artifacts define the tools & environment used in development.

It includes:

- Development tools (IDE, compiler)
- Configuration files
- Process guidelines
- Standards.

It provide a proper working environment for developers.

MANAGEMENT ARTIFACTS

[May [Dec, 2029]

Management artifacts are structured documents & outputs produced throughout the project lifecycle that help in:

- Decision making
- Tracking progress
- Managing risks
- Ensuring quality

Types of Management Artifacts -

1. Planning Artifacts

These are created at the beginning of the project.

Examples:-

Project Plan → Overall roadmap of the Project

Scope Statement → Defines what is included/excluded.

Schedule (Timeline) → Start & end dates of tasks.

Budget Plan → Cost Estimation

The purpose is to give clear direction and avoid confusion.

2. Monitoring & Control Artifacts

Used during project execution to track performance.

Example:-

Progress Report → Status of work completed

Milestone Report → Important checkpoints

Issue logs → Problems faced during project.

Risk Register → list of risk with solution.

It Ensure the project is on track & problems are handled early.

3. Communication Artifacts

Help in information sharing among stakeholders.

Example:-

Meeting minutes

Emails & Communication records

Stakeholders reports.

The Purpose is to maintain transparency & proper coordination.

4. Quality Artifacts

Ensures the project meets the required standards.

Examples:

Test Plans

Quality Checklists

Review Reports

The purpose is to maintain high quality & reduce errors.

5. Delivery Artifacts

These are the final outputs of the project.

Examples:

Final Product

User Documentation

Deployment Reports.

To deliver the completed project to the client.

6. Closure Artifacts

Prepared at the end of the project.

Examples:-

Project Closure Report

Lessons Learned Document

Final Review Report

Characteristics -

- Documented & structured
- Used throughout project lifecycle
- Help in decision - making.
- Improve communication & control.

ENGINEERING ARTIFACTS [Dec, 2024]

Engineering artifacts are the technical documents, models & outputs created during the development of a system or software.

They describe how the system is designed, built & works.

Basically, Engineering Artifacts are the 'Technical work' of the project (not planning, but actual development).

Types of Engineering Artifacts -

1. Requirement Artifacts

These define what the system should do.

Examples:-

Software Requirements Specification (SRS)

Use Case Diagrams

Functional & non-functional requirements.

To clearly understand user needs before development starts.

2. Design Artifacts

These explain how the system will be built.

Example:-

System Architecture Design

UML Diagrams (Class, Sequence, Activity).

Database Design.

The Purpose is to provide a blueprint for developers.

3. Implementation Artifacts

These are the actual development outputs.

Example:-

Source Code (C, C++ etc.)

Scripts & modules

Build files.

To create working system.

4. Testing Artifacts

These verify that the system works correctly.

Example:-

Test Cases

Test Plans

Bug Reports

Test Results

Purpose → To find & fix errors.

5. Deployment Artifacts

These are used to release the system.

Example:-

Executable files (.exe)

Installation packages

Configuration files.

Purpose → To make the system available to users.

6. Maintenance Artifacts

used after deployment for updates and improvements.

Examples:-

Patch updates
change requests
maintenance logs.

Purposes To keep the system updated and error-free.

Characteristics-

- Technical in nature.
- Created by developers & engineers
- Focus on system design & implementation.
- Evolve throughout the lifecycle.

Difference from Management Artifacts

Management Artifacts

Planning & control

Used by managers

Ex:- Project Plan

Engineering Artifacts

Development & design

Used by developers

Ex:- Source Code.

PRAGMATICS ARTIFACTS

[May, 2023]

Pragmatics artifacts are the practical, supportive & process-oriented documents / tools used in a project to make development efficient, organized & workable in real-world conditions.

They focus on "how work is actually carried out" rather than just planning (management) or building (engineering).

Types of Pragmatic Artifacts

1. Process Artifacts

They define how development activities are performed.

Examples:

Development guidelines

Coding standards.

Workflow / process descriptions

Purpose → To maintain consistency & discipline in work.

2. Configuration Management Artifacts

Used to manage changes & versions.

Example:-

Version control records (Git logs)

Configuration files.

Change history.

Purpose → To track changes & avoid confusion between versions.

3. Collaboration Artifacts

Help team members coordinates & work together.

Example:-

Task boards

Daily stand-up notes

Team communication logs.

Purpose → To improve teamwork & communication.

4. Review & Feedback Artifacts

Used to evaluate & improve work.

Example:-

Code review comments

Peer review notes

Feedback reports

Purpose → To improve quality through continuous feedback.

5. Tool & Environment Artifacts

Support the working environment of the project.

Example:

IDE configuration

Build tools setup

Automation scripts

Purpose → To make development faster and smoother.

6. Metrics & Tracking Artifacts

used to measure performance and productivity.

Example:

Velocity charts

Productivity reports

Purpose → To monitor efficiency & improve performance

Characteristics -

- Practical & real-world focused.
- Support daily project activities
- Improve efficiency & coordination
- Bridge gap between planning & development.

[May, 2023]

MODEL-BASED SOFTWARE ARCHITECTURE

[May, 2024]

Model-Based Software Architecture is a software design approach where the entire system is planned & described using models instead of directly written code.

A model is a simplified representation of a real system.

It helps developers understand:

- what the system will do.
- How it will behave
- How different parts are connected.

In model - Based Software Architecture :-

- Models are main artifacts (important outputs of the project).
- Code is developed after models are created.
- Models guide the entire development process.

It is used:

Writing code directly can create confusion and errors.

So, models are used to:

- Visualize the system
- Reduce complexity
- Plan before implementation

Example:-

If you are building a banking system:

Instead of coding directly,

You first create diagrams showing:

- Accounts
- Transactions
- User Interactions

These diagrams are known as models.

Role of models

1. As a Blueprint -

Models act like a map or plan before construction

2. As a Communication tool -

models help different people understands the System:

- Developers
- Project manager
- Clients

Even non-technical people can understand diagrams easily.

3. As Documentation

Models serve as permanent records of the system design.

Useful for:

- Future Maintenance
- New team members
- Upgrading the system

4. As a Problem - Solving Tool -

Before coding, models help identify:

- Design issue
- Missing requirements
- System flaws.

This reduce errors later.

Types of Models Used -

1. Structural Models

Structural model describe the static structure of the system.

They show -

- Components of the system
- Relationships between components.

Diagrams -

- Class Diagram
- Component Diagram
- Deployment Diagram

Example:

In a Shopping app:

Classes → User, Product, Order

Relationships → User places Order

2. Behavioral models-

Behavioral models describe how the system behaves over time.

They show:

- Flow of actions
- Interaction between components
- System response to events.

Common Diagrams:

- Sequence Diagram
- Activity Diagram
- State Diagram

Example:

Order process-

1. User selects product
2. Add to cart
3. makes payment

→ This shows how the system works step-by-step.

3. Architectural models-

Architectural models describes the overall structure & design of the system at a high level.

They show:

- System organization
- major components
- Communication between parts.

Common Architecture -

- Layered Architecture
- Client - Server model
- microservices Architecture

Example:-

In a web application-

- Frontend (UI Layer)
- Backend (Logic Layer)
- Database (data Layer)

This shows big picture of the system.

WORKFLOWS OF THE PROCESS

[June, 2025]
[May, 2022] [May, 2024]
[May, 2023] [Dec, 2024]

A workflow means a series of connected activities that are performed step-by-step to complete a project.

A workflow in Project management is a structured sequence of processes that guides a project from initiation to completion.

It includes initiation, planning, execution, monitoring & control & closing workflows like communication, risk & quality management.

These workflows ensure the project is completed efficiently on time & within budget.

The activities of the process are organized in to seven major ~~for~~ workflows:

- 1) Management
- 2) Environment
- 3) Requirement
- 4) Design
- 5) Implementation
- 6) Assessment
- 7) Deployment

1) Management workflow -

Control & manage the projects, plans activities & ensures everything runs smoothly.

2) Environment workflow -

Provide tools, software & setup needed for development.

3) Requirement workflow -

Collects & analyzes user needs & defines what the system should do.

4) Design workflow -

Creates the structure & blueprint of the system.

5) Implementation workflow -

Develop the actual software by writing code.

6) Assessment workflow -

Tests & checks the quality & performance of the system.

7) Deployment workflow -

Delivers the final product to users and makes it ready for use.

The Artifacts & Life - Cycle emphasises associated with each workflow

1) Management workflow

Artifacts :-

- Business case → why project is needed.
- Software development plan → how project will be done.
- Status assessment → progress report
- Vision → overall goal.
- Work Breakdown Structure → task division

Phases :-

Inception → Prepare business case & vision

Elaboration → Plan development

Construction → monitor & control development

Transition → monitor deployment

2. Environment workflow-

Artifacts:-

- Environment setup
- Software change order
- Database (for tracking changes)

Phase:-

Inception → Define tools & Environment

Elaboration → Install environment & setup database

Construction → maintain Environment & updates.

Transition → maintain Environment for support.

3. Requirements workflow-

Artifacts:-

- Requirement set
- Release specifications
- Vision.

Phase:-

Inception → Define system concept

Elaboration → Define architecture objectives

Construction → Define iteration goals

Transition → Refine release objectives

4) Design workflow -

Artifacts :-

- Design set
- Architecture description

Phase :-

Inception → Create architecture concept

Elaboration → Finalize architecture baseline

Construction → Design components

Transition → Improve design

5) Implementation workflow -

Artifacts :-

- Implementation set (code)
- Deployment set

Phase :-

Inception → Support prototype

Elaboration → Build architecture base

Construction → Develop complete system

Transition → maintain components

6) Assessment workflow -

Artifact:-

- Release specification
- Release descriptions
- User manuals
- Deployment set

Phase:-

Inception → Check plans & prototype

Elaboration → Evaluate architecture

Construction → Test internal releases

Transition → Test final product.

7) Deployment workflow -

Artifacts:-

- Deployment set

Phases:-

Inception → Study users

Elaboration → Prepare user manuals

Construction → Prepare release materials

Transition → Deliver product to users.

[June, 2025]

[May, 2022]

CHECKPOINTS OF THE PROCESS

[May, 2023]

[Dec, 2024]

Checkpoints are specific points in the process where progress is reviewed, decisions are made & the project is evaluated before moving forward.

They help ensure that the project stays on track, meets objectives & reduces risks.

Here are the main checkpoints of the process explained clearly:-

1. Major Milestones -

Major milestones are high level checkpoints that occur at the end of each major phase of a project lifecycle.

They are usually placed at the end of:

- Inception phase
- Elaboration Phase
- Construction Phase
- Transition Phase

Purpose -

- To evaluate whether the phase objectives are achieved.
- To check project feasibility & risks.
- To decide whether to continue, modify or stop the project.

2. Minor milestones -

Minor milestones are small checkpoints that occur at the end of each iteration or development cycle within a phase.

They occur inside each phase (especially in iterative models like Agile or RUP).

After completing a small part of work.

Purpose -

- To track short-term progress
- To ensure each iteration produces a working output.
- To identify & fix problem early.

3. Status Assessment -

Status Assessment is a regular & ongoing checkpoints used to monitor the current State of the Project.

They occurs - weekly, monthly or At regular intervals throughout the project.

Purpose -

- To compare planned progress vs actual progress.
- To identify delays, risks & issues
- To ensure the project is on track.